

Practical 6 – Problem sheet

This exercise sheet mainly practices input and output from files, and reading data from a web address. Exercise 4 returns to the topic of composite data types.

While working on this sheet, you should also practice writing Javadoc comments, as discussed in Lesson 13. Please see Exercises 1 and 2 in particular. *This will be relevant to the upcoming programming project.* If you are unsure about the Javadoc comments, please ask for feedback in the practical session.

Exercise 1 S

In this exercise, we consider the sequence of integers defined by

$$a_0 = 3, \quad a_{n+1} = 7 - 2a_n \quad (n \geq 0). \quad (1)$$

- (a) In the class `Sequence`, implement a procedure `writeToFile` that writes the numbers $a_0, \dots, a_n$ to a file (one number per line). The procedure should take two arguments: the file name (as `String`) and the number n (as `int`), where you can assume $n \geq 0$. Use `long` arithmetic.

In the procedure declaration, you can specify any exceptions that may occur (i.e., you don't need to handle any exceptions in your code).

Hints: For computing sequences of this type, see Practical 3, Exercise 2. For how to write to files, see Lesson 21, in particular the procedure `FileIO.writeFactorials`.

Note: Files that you write to the file system will be located in the same directory as your source code, unless specified otherwise.

- (b) For the code written in (a), write Javadoc comments to document the code. In particular:

- Add a Javadoc comment to the class `Sequence`.
- Add a Javadoc comment to the procedure `writeToFile`.
- Use `@param` tags to document all parameters of the procedure
- Use `@throws` tags to document exceptions thrown by the procedure.

Check the Javadoc output by selecting *Tools* → *Project documentation* from the BlueJ menu.

Exercise 2 I

This question follows up from Practical 4, Exercise 2. Recall that for a data series $x = (x_0, \dots, x_{n-1})$ of n real values, the following statistical quantities are defined:

the mean value,
$$\bar{x} := \frac{1}{n} \sum_{0 \leq j < n} x_j, \quad (2)$$

the k th moment (for $k \in \mathbb{N}$),
$$M_{k,x} := \frac{1}{n} \sum_{0 \leq j < n} x_j^k, \quad (3)$$

the standard deviation,
$$\sigma_x := \sqrt{\frac{1}{n-1} \sum_{0 \leq j < n} (x_j - \bar{x})^2}. \quad (4)$$

In this exercise, we write Java functions that read a list of floating point numbers *from a file*, and compute statistical quantities on this data series. Within the class `FileStats`, implement the following functions with parameter and return types as listed.

Name of function	Parameter types	Return type	Quantity computed
<code>meanValueFromFile</code>	<code>String</code>	<code>double</code>	$\bar{x}$
<code>momentFromFile</code>	<code>String, int</code>	<code>double</code>	$M_{k,x}$
<code>standardDeviationFromFile</code>	<code>String</code>	<code>double</code>	σ_x

Here x always refers to the data series read from the file, with the file containing one floating point number per line. The name of the file is given as a `String` parameter. In the case of $M_{k,x}$ the value of k is given by the `int` parameter.

These functions can (and should) throw the following exceptions:

- `java.io.IOException` if the file cannot be opened,
- `java.util.InputMismatchException` if the content of the file cannot be interpreted as floating point numbers.

As in Practical 4, the functions should return `Double.NaN` if the data series is too short to compute the relevant quantity.

After completing the functions, add a Javadoc comment to each of them, describing the purpose of the function, the parameters, the return value, the exceptions thrown, as well as the handling of any other exceptional input values.

Hint: Review the model solutions for Practical 4, which are available on Moodle. It may be a good idea to copy that code in and then modify it as required.

For testing you may want to create a test file with some numbers in it; this is best done in the same directory as the code.

Exercise 3 I

At the web address

<http://maths.york.ac.uk/static/progscicomp/dataseries-p6.txt>

you will find a data series of numbers between 0 and 1. Within the class `NetStats`, write the following functions, all of which take no arguments.

- Write a function `meanValueFromNet()`, which reads the data series (directly from the network address) and returns its mean value.
- Write a function `readDataSeries()`, which reads all entries of the data series and returns them in a `double[]` array. *Hint:* You will probably want to read the file twice: once for counting the number of lines, then for reading the entries.
- Using part (b), write a function `medianFromNet()` which computes the median of the data series. *Hints:* You need to sort the data series before computing the median; the class `java.util.Arrays` provides suitable functions for doing that (see its documentation). If you have forgotten the definition of the median, try an Internet search.

You can ignore the cases that the input file contains invalid numbers or too few numbers.

Exercise 4 (I)

In this exercise you will practice (once more) using composite data types.

- (a) Create a composite data type **Student** that contains information about a student. It should have the following fields, all of type **String**:
 - **firstName** to store the student's first name,
 - **lastName** to store the student's last name,
 - **emailAddress** to store the student's email address.
- (b) Write a function **StudentExample.testStudent** with no parameters that returns a **Student** record for the student “Joe Average” with e-mail address `joe.average@test.ac.uk`.
- (c) Write a procedure **StudentExample.sendWelcomeMessage** that takes a **Student** record as its input and sends a welcome e-mail to this student. Make use of the procedure **MockEmail.sendEmail** to that end.* The message to the student should have the subject “Welcome” and the following message text:

Dear <first name> <last name>, welcome to the autumn term.

Of course, “<first name>” should be replaced with the actual first name of the student, etc.

Please note: Your code should go into the classes **StudentExample** and **Student**. The class **MockEmail** is given and should not be modified.

Exercise 5 (A)

In this exercise, we will have a look at stream-based data processing. Specifically, suppose we are given a (potentially large) data file that contains nonnegative integers, one per line. We want to find which integer is the most frequent within the file. To that end, for efficiency reasons, we do not want to store the contents of the file in the computer's main memory, but rather read the data in a “stream” and process the numbers one-by-one, keeping track of how often we saw which number.

- (a) Write a function **mostFrequent** which takes a file name (as **String**) and an integer n , and returns the most frequent integer occurring in the corresponding file, assuming that all integers in the file are smaller than n .

It should work as follows: It keeps an `int[]` array of length n , where the j th element is used as a counter for how often the number j was seen in the file. These counters are updated as the file is read. In the end, one finds the number j with the largest counter.

- (b) The “naive” algorithm above needs a lot of memory for the counter array in case that n is large. However, if the distribution of numbers is “skewed” (i.e., there are a few numbers that occur very frequently, and most others occur very seldomly), then more efficient approximation algorithms can be used.

*Don't worry: the procedure doesn't actually send an e-mail. It prints a message to the screen instead. The class **MockEmail**, in which this procedure is provided, is given and should not be modified.

Here, let us consider the *SpaceSaver* algorithm[†], which works as follows, with an integer $k \geq 1$ given.

1. Initialize two arrays, called *label* and *counter*, with k entries, initially set to 0.
2. For all numbers e in the file:
 - 2.1 If e occurs in the array *label*, then increment the corresponding counter.
 - 2.2 Otherwise, find the element with the smallest counter, increment the counter, and replace the corresponding label with e .

At the end, *label* will contain the most frequently occurring numbers and *counter* the corresponding counters of occurrence (approximately). This assumes that k is “reasonably” large, but usually it can be much smaller than n in part (a).

Write a function `mostFrequentApprox` that approximates the most frequent number in the file by the above algorithm. The parameters should be the filename and k .

Hints and suggestions:

- Your code should go into the class `StreamCounter`.
- While the individual numbers in the file are in the usual `int` range, you should take care of the fact that the files to be read may contain very many numbers, hence you should use `long` values for all relevant counters.
- Three sample files are provided (small, medium and large size). See the static variables in the class `StreamCounter` for their names. (Note, you can directly type the filenames as `StreamCounter.sampleSmall` into the BlueJ input boxes, without quotation marks.) The “large” sample is > 1 GB in size and available on a network share, rather than being distributed with the code template; you need to be logged in on a classroom PCs to access it. Alternatively, you can download a compressed version at <http://maths.york.ac.uk/static/progscicomp/MostFreqSample-large.zip>; you then need to uncompress the file, and adapt the filenames in your program accordingly.
- The unit tests cover only the “small” example file. But the real test here is whether your code also works efficiently for the “large” file!
- You may initially want to use the class `java.util.Scanner` for reading the file contents, as discussed in the lectures. However, for very large files, it is much more efficient to use `java.io.BufferedReader` instead, which works a bit differently. You will need to look this up in the documentation! How much faster does the program run when using this class instead?

[†]see Metwally, A., Agrawal, D. and El Abbadi, A.: *Efficient computation of frequent and top-k elements in data streams*. In: Database Theory – ICDT 2005, vol. 3363 of Lecture Notes in Computer Science, pp. 398-412. Springer, 2005